Feedforward, Integral Action, Cascaded Loops and Windup

Contents

1. Overview . . . 2
2. Feedforward: the force the plan needs, before any error . . . 4
   - 2.1 Why feedback alone trails the plan . . . 4
   - 2.2 Feedforward from the robot's model . . . 6
   - 2.3 Feedforward plus feedback . . . 7
3. Integral action: zero error without a perfect model . . . 9
   - 3.1 State feedback needs the exact model . . . 9
   - 3.2 An integral state removes the error . . . 11
   - 3.3 The internal model principle . . . 12
4. Saturation and integrator windup . . . 14
   - 4.1 The actuator cannot follow every command . . . 14
   - 4.2 How the integral winds up . . . 15
5. Anti-windup: clamping and back-calculation . . . 17
   - 5.1 Clamping: stop integrating while saturated . . . 17
   - 5.2 Back-calculation: feed the shortfall back . . . 18
   - 5.3 The two methods on the held robot . . . 20
6. Motion control and force control: docking . . . 22
7. Rate limits and deadband on commands . . . 24
   - 7.1 Rate limits: never ask for more than the drive can give . . . 24
   - 7.2 Deadband: send nothing the motors cannot do . . . 25
8. Cascaded loops: a fast inner loop under a slower outer loop . . . 26
   - 8.1 One loop for the heading, or two . . . 27
   - 8.2 Inner loop first, then the outer loop . . . 28
   - 8.3 The inner loop catches disturbances first . . . 30
   - 8.4 Cascades in real stacks . . . 32
9. Summary . . . 34
10. Sources . . . 35
11. Key terms . . . 36

Where this fits:

- **Builds on:** Open-loop and closed-loop control; Regulation and tracking; PID control; Resonance and bandwidth.

1. Overview

Key point: A PI speed loop that works on paper fails on the robot because it waits for an error before it acts, keeps integrating while the drive sits at its 4 N limit, and has no place for a second sensor or a limit. Feedforward, integral action, anti-windup, force control, command limits and cascaded loops fix the six failures that follow.

The robot of the earlier chapters now has to drive its planned route, and its speed loop does the driving. The PI controller, the PID controller (G-2965) without its derivative term, turns the speed error into a drive force; the step response (G-2981) says how fast it settles; and the closed loop's poles (G-3000), the numbers that set how fast and how smoothly it settles, can be placed by the gains. The robot and the speed loop of Sections 2 to 7 (the heading loop of Section 8 has its own numbers there):

- m is the robot's mass, 8 kg;
- b is the rolling drag, 2 N · s/m: holding 0.5 m/s takes a force of 1 N;
- u is the drive force, at most 4 N either way;
- k_p is the PI's proportional gain, 20 N per m/s;
- k_i is its integral gain, 10 N per m (newtons per metre of the error's running total);
- the controller runs 100 times a second (100 Hz), and the top speed is 0.6 m/s.

The gains are those of RO-060's speed PI; on paper the loop holds 0.5 m/s and settles within a few seconds. Figure 1 shows what happens on a real floor when a person holds the robot for 3 s and then lets go. Watch the speed after the release: the robot does not return to 0.5 m/s but shoots up to 1.04 m/s.

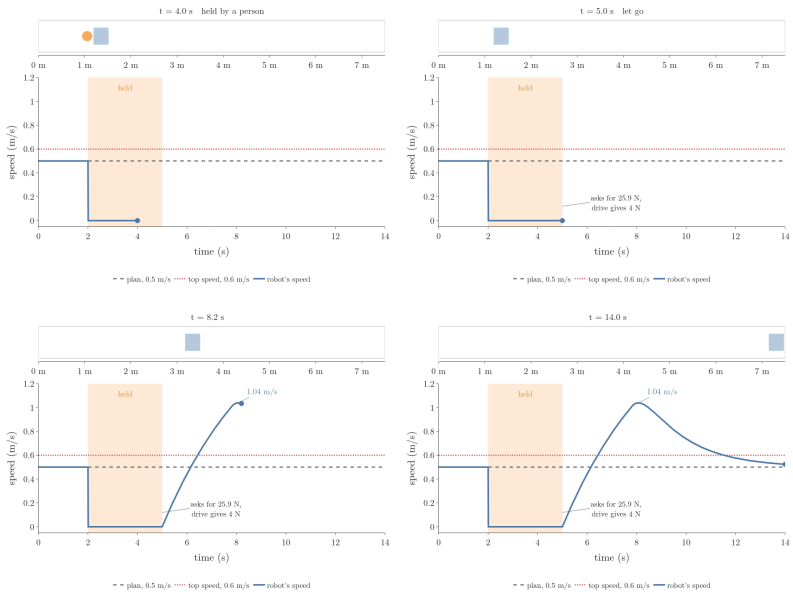


Figure 1: The robot cruising at 0.5 m/s under the PI speed loop, one frame per 0.2 s. Top: the robot in the corridor; the orange dot is the hand that holds it from t = 2 s to t = 5 s. Bottom: its speed (blue) against the plan, 0.5 m/s (grey dashed), and the top speed, 0.6 m/s (red dotted). At the release the PI asks for 25.9 N and the drive gives 4 N. The last frame shows the speed peaking at 1.04 m/s and staying above the top speed for 5.1 s

The held robot is one of six ways the loop fails on the delivery route, each worked out in the notebook:

- **It waits for an error.** Speeding up from rest to 0.5 m/s along the plan, the loop trails it by up to 0.085 m/s and ends 10 cm behind the plan for good.
- **It cannot cancel a repeating push.** At the dock's creep speed of 0.05 m/s, a wheel slightly out of round makes the speed swing between 0.027 and 0.073 m/s.
- **It winds up.** Held for 3 s, it asks for 25.9 N from a 4 N drive; let go, it reaches 1.04 m/s (Figure 1).
- **It pushes the dock too hard.** Against its charging dock it presses with the full 4 N instead of the 2 N the contacts want, and keeps pressing for 28 s after it is told to back away.
- **It gets commands it cannot follow.** A planner's jump from 0 to 0.5 m/s asks for 11 N at once, and near the goal the planner asks for speeds as small as 0.015 m/s that the motors cannot start.
- **Its heading loop gives way.** With RO-060's heading PID, a strip of carpet under one wheel swings the robot 11.1° off course, and the heading needs 10.6 s to come back within 0.5°.

The sections below fix them one by one, and Figure 2 shows where each fix sits in the controller:

- feedforward, the force the plan needs, computed before any error appears (Section 2);

- integral action, which removes a constant error whatever the model, and the internal model principle behind it (Section 3);
- the drive's limit and the integrator windup it causes (Section 4);
- anti-windup by clamping and by back-calculation (Section 5);
- force control for pressing against the dock (Section 6);
- rate limits and a deadband on the commands, as in Nav2's velocity smoother (Section 7);
- cascaded loops for the heading (Section 8).

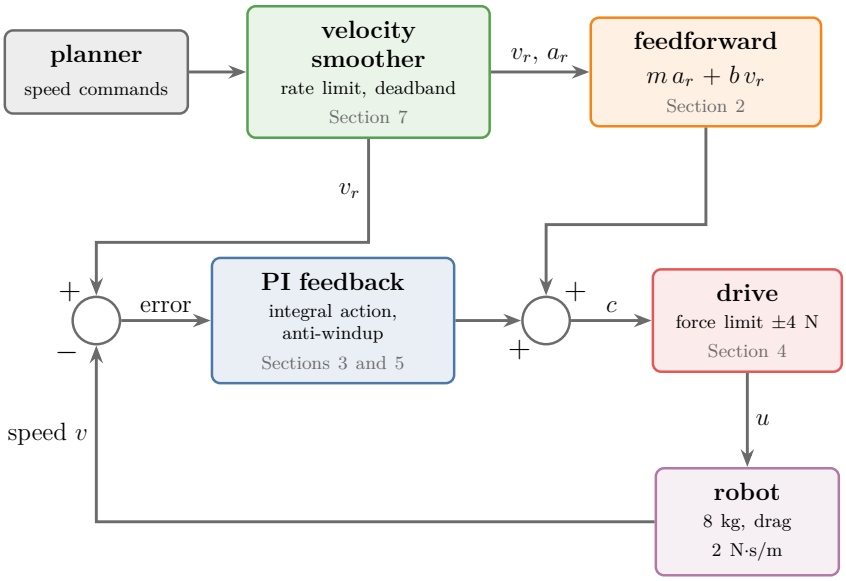


Figure 2: The speed controller this Note builds. The planner's speed commands pass the velocity smoother (rate limit and deadband, Section 7), which hands the wanted speed v_r and acceleration a_r on. The feedforward computes the force the plan needs, m a_r + b v_r (Section 2). The PI feedback, with integral action and anti-windup (Sections 3 and 5), turns the speed error into a correction. Their sum, the command c, passes the drive, which gives at most ±4 N (Section 4), as the force u that moves the robot; the robot's speed v is fed back

2. Feedforward: the force the plan needs, before any error

> **Key point:** Feedback acts only on an error, so it trails any plan that changes. Feedforward computes the force the plan needs from the robot's model, mass times acceleration plus drag, before any error appears; feedback then fixes only what the model gets wrong.

2.1 Why feedback alone trails the plan

The first failure of the Overview is the lag: 0.085 m/s behind while speeding up. The planner asks the robot to speed up from rest at 0.25 m/s^2 for 2 s, to 0.5 m/s, and then cruise. (Section 7 explains why the planner keeps speed-ups at 0.25 m/s^2, below what the drive can give.)

At the start the plan and the robot are both at rest, so the error is 0 and the PI's force is 0. The plan then speeds up, but the robot only gets a force once it has fallen behind and an error has built up. A feedback controller needs an error before the robot begins to move (MR §11.3.1.3). A driver who presses the accelerator only after seeing the car ahead pull away is in the same position.

Figure 3 plays the speed-up. Watch the red curve: feedback alone is 0.085 m/s behind the plan at t = 1.10 s. In the lower panel the distance behind the plan grows to 15.8 cm at t = 2.5 s and then settles at 10 cm, and stays there.

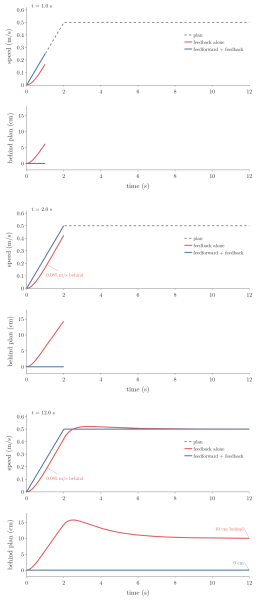


Figure 3: The planner's speed-up (grey dashed): from rest at 0.25 m/s^2 to 0.5 m/s in 2 s, then cruise, one frame per 0.2 s. Top: speed. Feedback alone (PI, red) is up to 0.085 m/s behind at t = 1.10 s; feedforward plus feedback (blue) sits on the plan. Bottom: how far each robot is behind where the plan says it should be. The last frame shows feedback alone 10 cm behind for good and feedforward plus feedback 0 cm behind

Why 10 cm, for good. At cruise the error is 0, so the proportional term is 0 and the integral term alone must supply the 1 N of drag. The integral term of integral action (G-2963) is the integral gain (G-2964) times the running total of the error over time. The speed error is the plan's speed minus the robot's, so its running total is the plan's distance minus the robot's: the gap. With:

- b, the drag, 2 N · s/m, and v, the cruise speed, 0.5 m/s;
- k_i, the integral gain, 10 N per m;
- x_{gap}, the gap behind the plan, in m.

Our numbers first:

$$2 \times 0.5 = 10 \times x_{\text{gap}}$$

$$x_{\text{gap}} = 1/10 = 0.1 \text{ m}$$

The same steps in symbols:

$$b\,v = k_i\,x_{\text{gap}}$$

$$x_{\text{gap}} = \frac{b\,v}{k_i}$$

The integral term can only reach the 1 N it needs by first collecting 10 cm of error, so the robot ends 10 cm behind the plan. A larger k_i shrinks the gap but never removes it, because the force still comes from an error. The fix is to supply the force without waiting for one. RO-059's speed loop already does so for one speed: its fixed push of 1 N is the drag at 0.5 m/s, sent before any error.

2.2 Feedforward from the robot's model

The plan already says, at every moment, the speed and the acceleration it wants. The robot's model says what force produces them: Newton's second law (G-2809), the net force equals mass times acceleration, plus the force that balances the drag. Computing the force from the plan and the model, before any error, and sending it to the drive is **feedforward** (G-3070) (FBS §1.2; MR §11.3.1).

One second into the speed-up, the plan's speed is 0.25 m/s and its acceleration 0.25 m/s^2. The force for the acceleration:

$$8 \times 0.25 = 2 \text{ N}$$

The force for the drag:

$$2 \times 0.25 = 0.5 \text{ N}$$

The feedforward at t = 1 s:

$$2 + 0.5 = 2.5 \text{ N}$$

At the end of the speed-up it is 3 N, and at cruise 1 N, the drag alone. With:

- m, the mass, 8 kg;
- a_r, the plan's acceleration, 0.25 m/s^2 during the speed-up, 0 at cruise;
- b, the drag, 2 N · s/m;
- v_r, the plan's speed, from 0 to 0.5 m/s;
- u_{ff}, the feedforward force, in N.

$$u_{\text{ff}} = m\,a_r + b\,v_r$$

Why it holds. The robot's speed changes by the net force divided by the mass: the drive force minus the drag. If the robot is exactly on the plan, its speed is v_r and its acceleration a_r; u is the drive force. At t = 1 s:

$$8 \times 0.25 = u - 2 \times 0.25$$

$$u = 2 + 0.5 = 2.5 \text{ N}$$

The same steps in symbols:

$$m\,a_r = u - b\,v_r$$

$$u = m\,a_r + b\,v_r$$

So the feedforward is the one force that keeps the robot on the plan. The feedforward runs the model backwards, from the wanted motion to the force; FBS §12.4 calls it an inverse model of the process. In Figure 3 the blue robot gets this force plus the PI's correction: it follows the plan exactly, its error stays 0, and the PI adds nothing. The lag of Section 2.1 is gone, but only because the model's 8 kg and 2 N · s/m are exact.

2.3 Feedforward plus feedback

No model is perfect. On carpet the real drag is 3 N · s/m, while the feedforward still uses the model's 2 N · s/m. Figure 4 repeats the speed-up on carpet. Watch the orange curve, feedforward alone: it gets 2 N · s/m worth of drag force where 3 are needed and drops to 0.336 m/s by t = 12 s, so it falls further behind every second, 140 cm by t = 12 s. The blue curve adds the PI: it stays within 0.015 m/s of the plan and is 4.98 cm behind at t = 12 s, still closing on 5 cm.

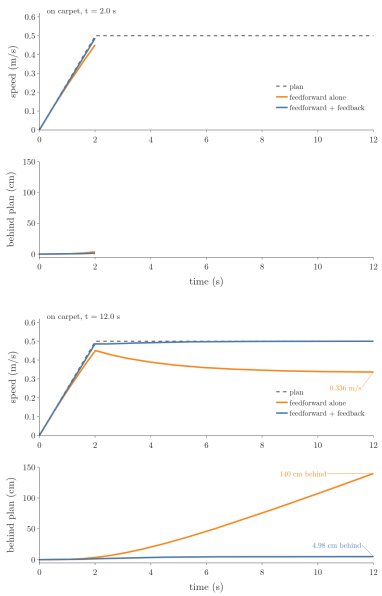


Figure 4: The same speed-up on carpet (real drag 3 N · s/m, the model says 2), one frame per 0.2 s. Top: speed against the plan (grey dashed). Feedforward alone (orange) drops to 0.336 m/s by t = 12 s; feedforward plus feedback (blue) stays within 0.015 m/s of the plan. Bottom: distance behind the plan. The last frame shows feedforward alone 140 cm behind and still falling back, feedforward plus feedback 4.98 cm behind

The two parts share the work. The feedforward supplies almost all the force, before any error; the feedback sees only what the model got wrong, here 0.5 N of extra drag, and its integral term collects that from a gap, as in Section 2.1:

$$0.5/10 = 0.05 \text{ m}$$

The controller of Figure 2 adds the two:

- k_p and k_i, the PI gains, 20 N per m/s and 10 N per m;
- e, the speed error, plan minus robot, in m/s;
- $\int e\,dt$, the running total of the error over time, the area under its curve, in m: an error of 0.1 m/s held for 2 s adds 0.2 m;
- c, the command sent to the drive, in N.

$$c = m\,a_r + b\,v_r + k_p\,e + k_i \int e\,dt$$

FBS §12.4 gives the reason for splitting the work: the feedback can then be designed for robustness and disturbances, and the feedforward separately for following the plan. MR §11.3.1.3 calls feedforward plus PI feedback its preferred control law for a robot joint.

A disturbance we can measure. A ramp's pull can be fed forward too. We take a 1° ramp, steeper than the ramp of feedback basics, which rises 1 cm per metre. Gravity pulls

the 8 kg robot back along it with the weight times the sine of the slope:

$$\sin 1^\circ = 0.01745$$

$$8 \times 9.81 \times 0.01745 = 1.37 \text{ N}$$

A tilt sensor that reads 1° lets the controller add those 1.37 N before the speed drops. Feeding a measured disturbance forward is **disturbance feedforward** (G-3071) (FBS §12.4). Disturbance feedforward needs a disturbance that can be measured: a ramp's slope can be, a gust of wind on a car cannot (MATLAB, What Is Feedforward Control?).

In real stacks. Autoware's longitudinal controller (`control/autoware_pid_longitudinal_controller`, in `pid_longitudinal_controller.cpp`) sends the planned acceleration plus a slope term as feedforward and lets a PID on the speed error remove what is left; its README says the feedforward alone should be enough "under ideal conditions with no modeling error". Apollo's longitudinal controller (`modules/control/controllers/lon_based_pid_controller/lon_controller.cc`) adds the speed PID's output, the planned ("preview") acceleration and a slope offset. PX4's position controller (`src/modules/mc_pos_control/PositionControl/PositionControl.cpp`) adds the planned velocity and acceleration to its loops, and ros2_controllers' `pid_controller` has a `feedforward_gain` parameter.

Feedforward plus feedback now follows the plan, but on carpet the speed came back to 0.5 m/s only through the integral term. The next section asks why an integral removes a constant error whatever the model, and what it cannot remove: the wheel's repeating push.

3. Integral action: zero error without a perfect model

> **Key point:** A controller that feeds back the state lands on the wanted speed only if its model is exact; adding the integral of the error as an extra state forces the error to zero for any model and any constant disturbance. To cancel a disturbance of another shape, the controller must contain a model of that shape: the internal model principle.

3.1 State feedback needs the exact model

The carpet run of Section 2.3 needed the integral; here is what happens without one. At cruise the plan's acceleration is 0, so the controller of Section 2.3 (its symbols are listed there) without its integral term sets the drive force to:

$$u = b\, v_r + k_p\,(v_r - v)$$

$$u = -20\, v + 22\, v_r$$

The **state** (G-2314) is what a model keeps track of because it decides what happens next. For the speed model, given the force, one number decides the future: the speed v. A controller that multiplies the state by a gain and adds a gain times the reference is **state feedback** (G-3072) (FBS §7.2). With:

- v, the speed, the state, in m/s;
- v_r, the reference, the wanted speed, 0.5 m/s;
- k, the feedback gain on the state, 20 N per m/s;
- k_f, the gain on the reference, 22 N per m/s, chosen from the model so that the speed lands on the reference.

$$u = -k\,v + k_f\,v_r$$

At 0.5 m/s it gives the drag force the model expects:

$$u = -20 \times 0.5 + 22 \times 0.5 = 1 \text{ N}$$

On carpet the drag at 0.5 m/s is 1.5 N, not 1 N, and the speed settles where the force equals the real drag. Our numbers first, with the real drag 3 N · s/m:

$$-20\,v + 22 \times 0.5 = 3\,v$$

$$11 = 23\,v$$

$$v = 11/23 = 0.478 \text{ m/s}$$

On a 1° ramp the drag is 2 N · s/m but gravity pulls back with 1.37 N:

$$-20\,v + 11 = 2\,v + 1.37$$

$$v = 9.63/22 = 0.438 \text{ m/s}$$

The same steps in symbols, with b the real drag and d the ramp's pull:

$$-k\,v + k_f\,v_r = b\,v + d$$

$$v = \frac{k_f\,v_r - d}{k + b}$$

The speed equals v_r only if k_f equals $k + b$ with the true b and d is 0. A gain chosen from the model is right only for the floor the model describes. The red curve of Figure 5 shows the ramp case: state feedback settles 0.062 m/s too slow.

3.2 An integral state removes the error

The fix is to give the controller a second state of its own: the running total of the error. With:

- z, the integral state, the running total of the speed error over time, in m;
- k_i, its gain, 10 N per m.

$$\frac{dz}{dt} = v_r - v$$

$$u = -k\,v + k_f\,v_r + k_i\,z$$

Why the error must vanish. If the loop settles, every state stops changing, z included. The integral state stops changing only when its rate, the error, is zero:

$$\frac{dz}{dt} = 0$$

$$v_r - v = 0$$

$$v = v_r$$

Nothing in these three lines uses the drag, the ramp or the gains: the speed lands on the reference "independently of the specific values" of the model, as long as the loop is stable (FBS §7.4). Adding the integral of the error as an extra state of the controller is **integral action in state feedback** (G-3073) (FBS §7.4).

Figure 5 plays the 1° ramp. Watch the integral term in the lower panel: it climbs while the robot is slow and stops at 1.37 N, the ramp's pull, at the moment the error reaches zero. The speed (blue) dips to 0.451 m/s and is back within 0.005 m/s of 0.5 m/s after 5.32 s. On carpet the integral term stops at 0.5 N, the extra drag.

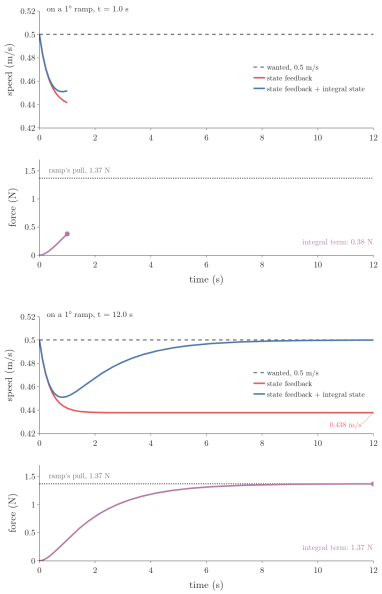


Figure 5: State feedback on a 1° ramp, with and without the integral state; the robot meets the ramp at t = 0 cruising at 0.5 m/s, one frame per 0.2 s. Top: speed against the wanted 0.5 m/s (grey dashed). State feedback alone (red) settles at 0.438 m/s; with the integral state (blue) it dips to 0.451 m/s and comes back. Bottom: the integral term of the second controller (purple), climbing to the ramp's pull, 1.37 N (grey dotted). The last frame shows the integral term at 1.37 N and the speed at 0.5 m/s

With k_f equal to k, this controller is exactly the PI controller of Section 1 written in another way:

$$u = k\,(v_r - v) + k_i\,z$$

FBS §11.1 gives another way to see the integral term: it generates the feedforward force by itself. Here it learned the 1.37 N a ramp feedforward would have sent, without a tilt sensor. FBS §11.1 adds that zero steady-state error is better achieved by integral action than by feedforward, which needs exact process parameters; Section 2.3 used both. What an integral cannot do is cancel a push that keeps changing.

3.3 The internal model principle

The integral removes any constant push. The wheel's push in the Overview is not constant. At the dock the robot creeps at 0.05 m/s; its wheels, of radius 0.05 m (rolling without slipping (G-2290)), then turn at 1 rad/s, once every 6.28 s. A wheel slightly out of round pushes and drags the robot once per turn; we take a push that swings 0.5 N each way:

- $d(t)$, the wheel's push at time t, in N, with t in s.

$$d(t) = 0.5\,\sin(t)$$

The red curve of Figure 6 shows the PI's answer: the speed swings between 0.027 and 0.073 m/s, for ever, a swing of 0.023 m/s each way around 0.05 m/s.

Why the integral cannot cancel it. In a settled loop the error is 0, so whatever cancels the push must keep producing it with no error coming in. An integrator with no input holds its last value, a constant, so it can cancel a constant push and nothing else. A push that swings at 1 rad/s needs a part that keeps swinging at 1 rad/s with no input: an oscillator, like a pendulum that keeps swinging once pushed. The controller must contain a model of the signal it has to cancel; this is the **internal model principle** (G-3074): a constant needs an integrator, a sine of frequency 1 rad/s needs an oscillator at 1 rad/s (Schenato, lectures 14 and 15; FBS §8.5).

A controller part that oscillates by itself at a chosen frequency and is driven by the error is a **resonant term** (G-3075). With:

- q, the resonant term's own state, in m · s;
- ω_0, its frequency, 1 rad/s, the wheel's turn rate;
- e, the speed error, in m/s;
- k_{r1} and k_{r2}, its two gains, 6 N per m · s and 17.5 N per m.

$$\frac{d^2q}{dt^2} = -\omega_0^2\, q + e$$

$$u_{\text{res}} = k_{r1}\, q + k_{r2}\, \frac{dq}{dt}$$

With no error coming in, the first line describes a swing at 1 rad/s that keeps its size. The two gains were chosen in the notebook so that the loop stays stable and settles within about 10 s, and u_{res} is added to the PI's command.

Figure 6 runs both controllers. Watch the lower panel: the resonant term's output (blue dashed) grows into a 0.5 N wave that mirrors the wheel's push (orange). The two cancel, and the blue speed in the upper panel settles on 0.05 m/s; after 20 s it swings by less than 0.00001 m/s.

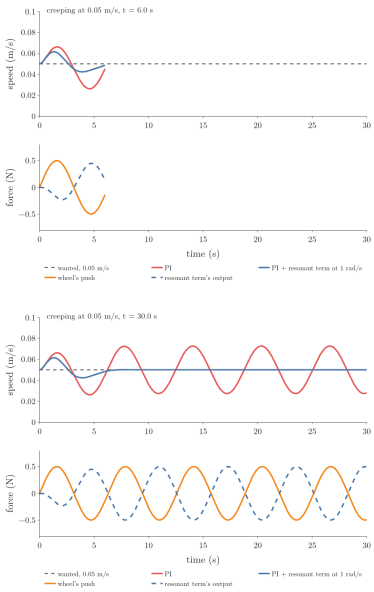


Figure 6: Creeping at 0.05 m/s with an out-of-round wheel pushing 0.5 sin(t) N, one frame per 0.3 s for 30 s. Top: speed. PI alone (red) swings between 0.027 and 0.073 m/s; PI plus a resonant term at 1 rad/s (blue) settles on 0.05 m/s (grey dashed). Bottom: the wheel's push (orange) and the resonant term's output (blue dashed). The last frame shows the resonant term's output as a 0.5 N wave that mirrors the push

Integral action and its generalisation remove a disturbance whatever the model, but each needs the drive to deliver whatever force the integral asks for. The next section shows what happens when it cannot.

4. Saturation and integrator windup

> **Key point:** The drive gives at most 4 N. While it sits at that limit the error stays, so the integral keeps growing; when the error changes sign, the oversized integral must first come back down, and meanwhile the drive keeps pushing. Integrator windup caused the 1.04 m/s of Figure 1.

4.1 The actuator cannot follow every command

The integral of Section 3 assumed the drive delivers any force it is asked for. The plant (G-2945), the system being controlled, is really two parts (Figure 7). The actuator (G-2948), here the drive, makes the force; the process, here the robot's body, responds to it.

A linear model has no limits: ask for 25.9 N and the model's drive gives 25.9 N. The real drive, like every real actuator, has a ceiling: a motor can only spin so fast (FBS §11.4). Ours gives at most 4 N either way, an acceleration of at most 0.5 m/s^2 from rest. An actuator stuck at its limit while the command asks for more is **actuator saturation** (G-3076). While

the drive is saturated, the force no longer depends on the error, so the feedback loop is broken and the robot runs open loop (FBS §11.4).

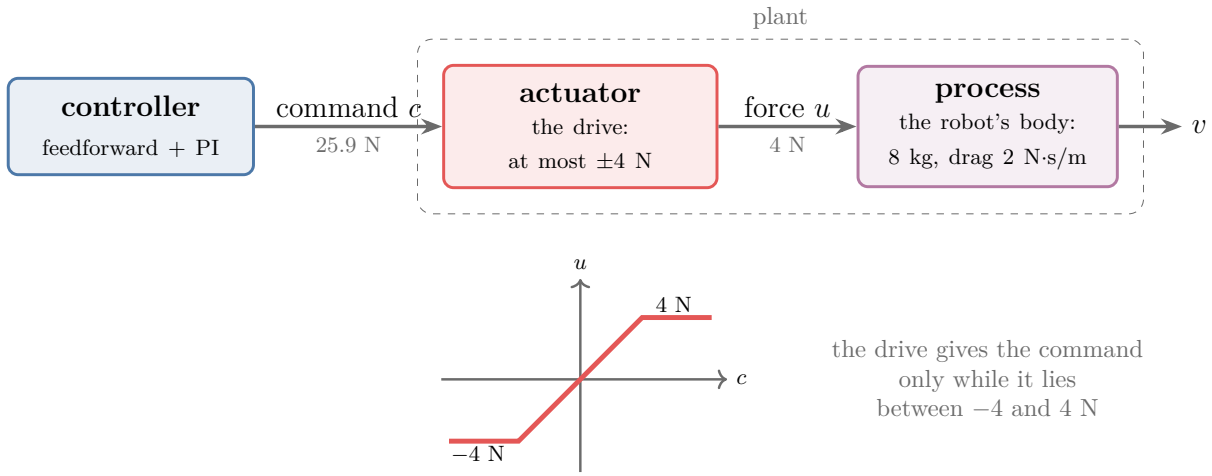


Figure 7: The plant split in two. The controller (feedforward plus PI) sends the command c to the actuator, the drive, which gives at most ±4 N; the drive's force u moves the process, the robot's body (8 kg, drag 2 N · s/m), whose speed is v. The numbers are those at the moment the held robot of Figure 1 is let go: c = 25.9 N, u = 4 N. Below: the drive's force against the command, a straight line that flattens at 4 N and −4 N

4.2 How the integral winds up

Figure 8 below shows the held robot of Figure 1 from inside the controller. The robot cruises at 0.5 m/s with feedforward plus PI, so the feedforward supplies the 1 N of drag and the integral term sits at 0 N. At t = 2 s a person holds it still. The error is now 0.5 m/s, and the integral term grows by the integral gain times the error every second:

$$10 \times 0.5 = 5 \text{ N per s}$$

$$5 \times 3 = 15 \text{ N after 3 s}$$

(The simulation starts adding at the first 100 Hz step after the grab and gives 14.90 N.) At the release the command is the feedforward plus the proportional term plus the integral term:

$$c = 1 + 20 \times 0.5 + 14.90$$

$$= 1 + 10 + 14.90 = 25.90 \text{ N}$$

The drive gives 4 N. The integral has no way of knowing that the drive gave up long ago, so it kept growing (FBS §11.4).

Figure 8 replays the run inside the controller. Watch the dashed command and the integral term (purple) climb while the robot is held, and the drive's force (solid red) stay flat at 4 N. Let go, the robot speeds up and passes 0.5 m/s at t = 6.15 s, and the error turns negative. The integral term now shrinks, but from 17.7 N, so the command stays above 4 N and the drive keeps pushing with its full force until t = 7.82 s. By then the robot is at 1.01 m/s; it peaks at 1.04 m/s at t = 8.07 s and stays above its 0.6 m/s top speed for 5.1 s.

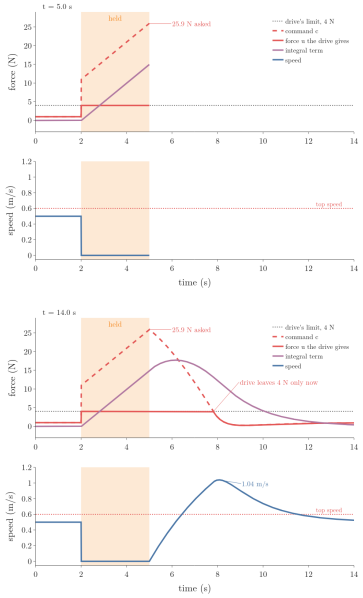


Figure 8: The held robot inside the controller (feedforward plus PI), one frame per 0.2 s; the orange band is the 3 s of holding. Top: the command c (red dashed), the force u the drive gives (red solid, at most 4 N, grey dotted) and the integral term (purple). While held, the integral term climbs to 14.9 N and the command to 25.9 N. Bottom: speed. The last frame shows the drive at 4 N until 7.82 s and the speed peaking at 1.04 m/s, above the top speed (red dotted)

An integral that keeps growing while the actuator is saturated, and then makes the loop overshoot while it unwinds, is **integrator windup** (G-3077) (FBS §11.4). FBS Example 11.5 shows the same thing on a car whose throttle saturates on a 6° hill: the speed settles 15 s after the error first reaches zero, with a large overshoot. Asking for more than 4 N does no harm by itself; the harm comes from undoing the extra integral afterwards (MATLAB, Anti-windup for PID control). So the fix is to keep the integral from growing in the first place.

5. Anti-windup: clamping and back-calculation

> **Key point:** Anti-windup stops the integral from growing while the drive is saturated, so the command leaves the limit as soon as the error changes sign. Clamping switches the integration off; back-calculation feeds the gap between the force given and the force asked into the integrator.

Any change to the controller that stops integrator windup is **anti-windup** (G-3078). The held robot of Section 4 reached 1.04 m/s; of the two common methods below, clamping brings the peak to 0.515 m/s and back-calculation removes the overshoot.

5.1 Clamping: stop integrating while saturated

The first step limits the controller's own output to what the drive can give, ±4 N, so the controller knows when it is saturated (`control_toolbox`, `pid.cpp`). Clamping then makes two checks at every step (Figure 9):

1. **Is the output saturated?** Compare the command before and after the limit; if they differ, yes.
2. **Is the integrator making it worse?** If the error and the command have the same sign, both positive or both negative, the integrator is pushing the command further past the limit.

If both answers are yes, a switch sets the integrator's input to 0, and integration stops. Once the error changes sign, or the output leaves the limit, the input is restored. Stopping the integration while the output is saturated and the error has the same sign as the command is **clamping** (G-3079), also called conditional integration; ros2_control's `control_toolbox` runs exactly these two checks under the name `conditional_integration` (`pid.cpp`).

Some sources and stacks use "clamping" for a different fix: holding the integral term's value inside fixed bounds, as in Beauregard's "Reset Windup" post, Autoware's `max_i_effort` and Apollo's `integrator_saturation_level`. In this Note, clamping always means stopping the integration; the other fix is called limiting the integral term.

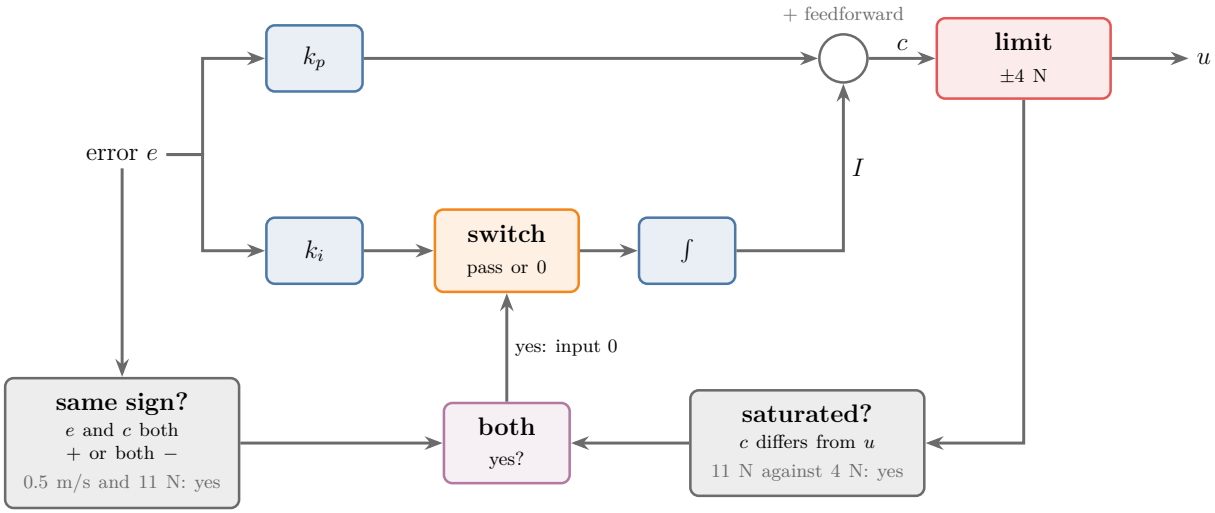


Figure 9: Clamping. The error e goes to the proportional gain k_p and, through the integral gain k_i and a switch, to the integrator, whose output I joins the sum with the proportional term and the feedforward; the command c passes a ±4 N limit as u. Two checks feed the switch: "saturated?" (c differs from u: 11 N against 4 N while the robot is held, yes) and "same sign?" (e and c both positive: 0.5 m/s and 11 N, yes). When both answer yes, the switch sets the integrator's input to 0

On the held robot the two checks answer yes from the first step. The command is the feedforward plus the proportional term plus the integral term:

$$c = 1 + 10 + 0 = 11 \text{ N}$$

The drive gives 4 N, and the error, 0.5 m/s, has the same sign as the 11 N. The integral term stays at 0 N for the 3 s. At the release the command is 11 N, still saturated, so the integral stays frozen while the robot speeds up. At t = 5.77 s, at 0.35 m/s, the proportional term no longer fills the drive, the command drops below 4 N and integration resumes. By the time the robot passes 0.5 m/s the integral term has grown only to 0.43 N, against 17.7 N without anti-windup, and the robot peaks at 0.515 m/s.

Set the limit a little low. The real limit falls as the battery runs down, the motors warm up or the wheels wear. If the controller's limit is set at exactly 4 N and the drive can only give 3.5 N, the controller does not notice the saturation between 3.5 and 4 N, and the integral winds up by the difference. A controller limit a little below the drive's, such as 3.8 N, leaves room for this (MATLAB, Anti-windup for PID control).

5.2 Back-calculation: feed the shortfall back

Back-calculation does not switch the integrator off. Instead it measures how far the command overshoots the drive's limit and feeds that shortfall into the integrator, which pulls the command back to the limit (FBS §11.4, Figure 11.11). With:

- I, the integral term, in N;
- k_i, the integral gain, 10 N per m, and e, the speed error, in m/s;
- c, the command, and u, the force the drive gives, both in N;
- k_{aw}, the back-calculation gain, 1 per s, the best of the three values the notebook tries (the end of this section compares 2 per s and 0.5 per s).

$$\frac{dI}{dt} = k_i\, e + k_{aw}\,(u - c)$$

When the drive is not saturated, u equals c and the second term is 0, so the integrator works as before. Feeding the gap between the force the drive gives and the command into the integrator through a gain is **back-calculation** (G-3080).

While the robot is held, the integral term stops changing when the two parts of its rate cancel (Figure 10). The error's part:

$$10 \times 0.5 = 5 \text{ N per s}$$

The shortfall's part must then be -5 N per s:

$$1 \times (4 - c) = -5$$

$$c = 9 \text{ N}$$

The command settles 5 N above the limit. The proportional and feedforward terms give 11 N of it, so the integral term heads for:

$$9 - 11 = -2 \text{ N}$$

The same steps in symbols, with the rate set to 0:

$$0 = k_i\, e + k_{aw}\,(u - c)$$

$$c = u + \frac{k_i\, e}{k_{aw}}$$

The integral term has reached -1.9 N by the release, so the command leaves the limit at t = 5.63 s, and the robot comes up to 0.5 m/s without overshooting; it is within 0.01 m/s of 0.5 m/s after 7.43 s. A larger k_{aw} keeps the command closer to the limit and resets it faster, but measurement noise limits how large it can be (FBS §11.4), and too large a gain pulls the integral term too far down: at 2 per s it reaches -4.5 N and the robot creeps up from

below, within 0.01 m/s only after 10.34 s. ros2_control describes the gain by its tracking time constant, T_t, one divided by k_{aw}, and by default sets it from the PI gains:

$$T_t = k_p/k_i = 20/10 = 2 \text{ s}$$

$$k_{aw} = 1/2 = 0.5 \text{ per s}$$

With that default the robot reaches 0.594 m/s, just under the top speed, and is within 0.01 m/s only after 11.39 s: the gain needs choosing.

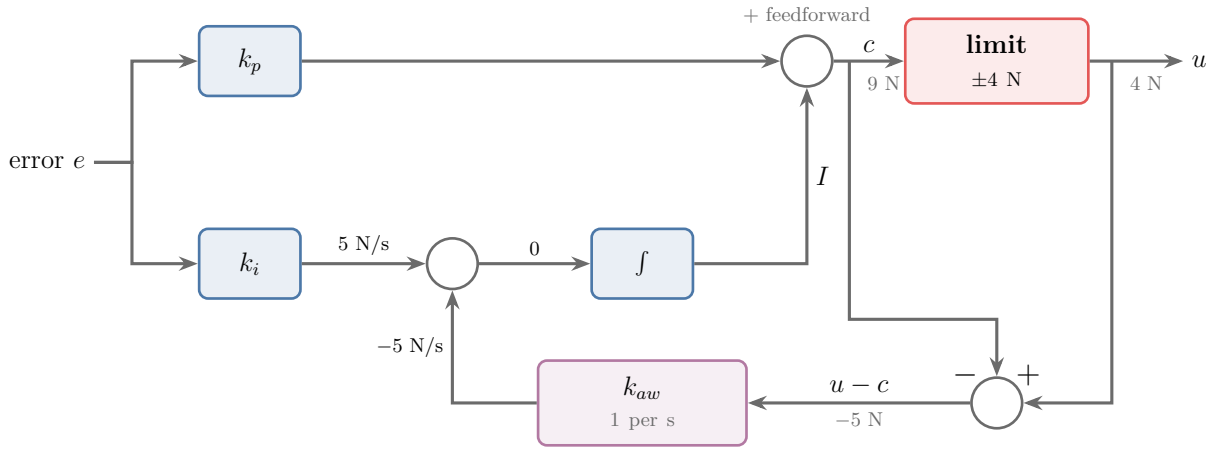


Figure 10: Back-calculation, with the numbers while the robot is held and k_aw = 1 per s. The error e passes the proportional gain k_p to the sum and, through the integral gain k_i (5 N/s), to an adder before the integrator. The command c (9 N) passes the ±4 N limit as u (4 N). The gap u − c (−5 N) passes the gain k_aw (1 per s) and enters the adder as −5 N/s, so the integrator's input is 0

5.3 The two methods on the held robot

Figure 11 runs the held robot with no anti-windup, with clamping and with back-calculation. Watch the integral terms during the 3 s of holding: the red one climbs to 14.9 N, the blue one (clamping) stays at 0 N, the green one (back-calculation) falls to −1.9 N. After the release the red speed peaks at 1.04 m/s and the blue at 0.515 m/s, and the green comes up to 0.5 m/s without overshooting. Clamping is within 0.01 m/s of 0.5 m/s after 8.62 s, back-calculation after 7.43 s.

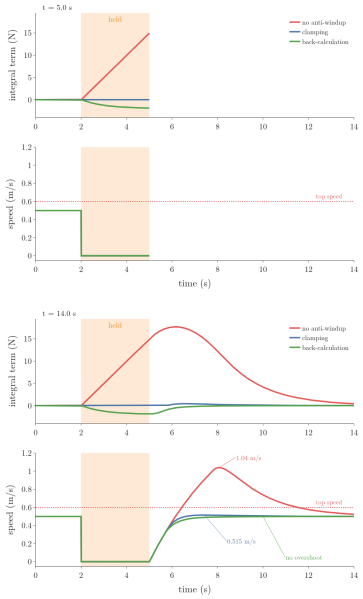


Figure 11: The held robot with three controllers, one frame per 0.2 s; the orange band is the 3 s of holding. Top: the integral term without anti-windup (red, up to 14.9 N), with clamping (blue, stays at 0 N) and with back-calculation, k_aw = 1 per s (green, falls to −1.9 N). Bottom: speed. The last frame shows the peaks: 1.04 m/s without anti-windup, 0.515 m/s with clamping, and no overshoot with back-calculation; the top speed is red dotted

In real stacks. Every stack in this chapter's list has anti-windup:

- **PX4** (`src/modules/mc_pos_control/PositionControl/PositionControl.cpp`): the vertical velocity loop uses clamping (it zeroes the error fed to the integrator when the thrust is at its limit and the error pushes further), the horizontal loop uses back-calculation ("tracking anti-windup") with a gain of 2 divided by the proportional gain; the body-rate loop (`src/lib/rate_control/rate_control.cpp`) stops integrating in a saturated direction.
- **Apollo** (`modules/control/control_component/controller_task_base/common/`): `pid_IC_controller.cc` ("integral clamping") and `pid_BC_controller.cc` ("backward calculation", with a gain `kaw`), besides the plain `pid_controller.cc`, which limits the integral term's value.
- **ros2_control** (`control_toolbox`, `pid.hpp` and `pid.cpp`): the parameter `antiwindup_strategy` takes `back_calculation` or `conditional_integration`; `tracking_time_constant` sets the back-calculation, with a default, for a PI controller, of the proportional gain divided by the integral gain.
- **Autoware** (`autoware_pid_longitudinal_controller`, `pid.cpp` and its parameter file): limits the integral term to ± 0.3 m/s^2 (`max_i_effort`) and, by default, does not integrate while the car moves slower than 0.5 m/s, so that a car held still by another system does not wind up, the held robot of Section 4 exactly.

Anti-windup lets the held robot recover. A robot that pushes against something that never moves is a different case: its error never changes sign.

6. Motion control and force control: docking

> **Key point:** When the task is to press, not to move, a speed loop is the wrong tool: its speed error never goes away, so it pushes as hard as the drive allows. Force control sets the pushing force itself.

The fourth failure of the Overview: at the charging dock the robot presses with the full 4 N, twice the 2 N the contacts want. The robot creeps at 0.05 m/s into a dock that does not move and touches it at t = 2 s. The speed loop still wants 0.05 m/s and the robot stands still, so the speed error stays at 0.05 m/s. Even with clamping (no integrating while the drive is saturated), the integral term grows every second by the integral gain times that error:

$$10 \times 0.05 = 0.5 \text{ N per s}$$

The command reaches the 4 N limit at t = 7.8 s; there clamping stops the integral, and the robot keeps pressing with 4 N. At t = 32 s the plan says back away at 0.05 m/s: the integral first has to shrink, and the robot keeps pressing for 3.66 s. Without anti-windup the integral keeps climbing at the dock, to 15 N after 30 s, and the robot presses on for 27.85 s after the back-away command.

Controlling where the robot goes and how fast, as every loop so far does, is **motion control** (G-3081). When the task is not to move but to apply a force, force control is needed (MR §11.5). Setting the force on the environment directly, and measuring it if a sensor is there, is **force control** (G-3082). Ideal force control keeps the force the same whatever motion the environment makes, just as ideal motion control keeps the motion the same whatever forces push on the robot (MR §11.5).

Why the drive force is the contact force. Against the dock the robot stands still: its speed and its acceleration are both 0. Newton's second law then says the drive force minus the drag minus the dock's push is 0. With:

- m, the mass, 8 kg, and b, the drag, 2 N · s/m;
- a and v, the robot's acceleration and speed, both 0 against the dock;
- u, the drive force, the wanted 2 N;
- F, the dock's push on the robot, in N.

Our numbers first:

$$8 \times 0 = 2 - 2 \times 0 - F$$

$$F = 2 \text{ N}$$

The same in symbols:

$$m\,a = u - b\,v - F$$

$$F = u \quad \text{when } a = v = 0$$

So once the bumper detects contact, the robot switches from motion control to force control and sets the drive force to 2 N. Figure 12 runs both robots. Watch the force panel: the motion-controlled robot's force climbs to 4 N and stays; the force-controlled robot holds 2 N, and at the back-away command it leaves the dock within one step.

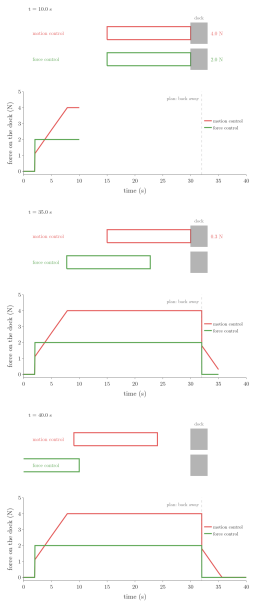


Figure 12: Docking with motion control (red, feedforward plus PI with clamping) and with force control (green), one frame per 0.5 s. Top: the two robots (0.3 m long) and the dock, with the force on the dock. Bottom: that force over time; the grey dashed line marks the plan's back-away command at t = 32 s. Motion control pushes harder and harder until the drive's 4 N limit at t = 7.8 s and leaves the dock 3.66 s after the command; force control holds 2 N and leaves at once

Setting the force from the model alone trusts that nothing else pushes: friction in the wheels' gears would take part of the 2 N. With a force sensor in the bumper, a PI on the force error, plus the wanted force as feedforward, removes that error the same way the speed loop removed the drag error (MR §11.5, equation 11.51).

Force control fixes the dock. The saturations so far started with what the robot was asked to do: a speed it could not reach while held, a speed it could not reach against the dock. Many saturations start in the commands themselves.

7. Rate limits and deadband on commands

> **Key point:** A jump in the wanted speed asks for more force than the drive has, and a very small wanted speed asks for a motion the motors cannot start. A velocity smoother limits how fast the command may change and sends 0 for commands under a deadband, so the speed loop never receives a command it cannot follow.

7.1 Rate limits: never ask for more than the drive can give

The fifth failure of the Overview: a planner's jump from 0 to 0.5 m/s asks for 11 N at once. The planner does not always send smooth profiles; a planner that recomputes 20 times a second may switch from "stop" to "go at 0.5 m/s" between two cycles. The speed error jumps to 0.5 m/s, and the command is the feedforward plus the proportional term:

$$c = 2 \times 0.5 + 20 \times 0.5$$

$$= 1 + 10 = 11 \text{ N}$$

The drive gives 4 N and stays saturated for 0.77 s (and again for 0.48 s when the planner's command drops at 3.5 s), and every saturation invites windup, the integral growing while the drive is at its limit (Section 4). The cure is to limit how fast the command may change. Nav2's velocity smoother does it in a loop that runs at a fixed rate, 20 times a second by default (`nav2_velocity_smoother`, `velocity_smoother.cpp`). Each cycle it moves its output towards the planner's command, but by no more than the acceleration limit divided by the rate:

$$0.25/20 = 0.0125 \text{ m/s per cycle}$$

A limit on how fast a signal may change per second is a **rate limiter** (G-3083). With:

- v_{cmd}, the planner's command this cycle, in m/s;
- v_{last}, the smoother's output in the last cycle, in m/s;
- $a_{\max}$, the acceleration limit, 0.25 m/s^2;
- f, the smoother's rate, 20 per s;
- L, the largest change per cycle, the acceleration limit divided by the rate, 0.0125 m/s;
- Δ, the change the planner asks for this cycle, in m/s;
- $\text{clip}(\Delta, -L, L)$, the change cut back to $-L$ if it is below $-L$ and to L if it is above L;
- v_{out}, the smoother's output this cycle, in m/s.

$$L = a_{\max}/f$$

$$\Delta = v_{\text{cmd}} - v_{\text{last}}$$

$$v_{\text{out}} = v_{\text{last}} + \text{clip}(\Delta, -L, L)$$

The clip keeps each change between -0.0125 and 0.0125 m/s. Starting from rest with a command of 0.5 m/s, the first three outputs:

$$0 + 0.0125 = 0.0125 \text{ m/s}$$

$$0.0125 + 0.0125 = 0.025 \text{ m/s}$$

$$0.025 + 0.0125 = 0.0375 \text{ m/s}$$

After 40 cycles, 2 s, the output reaches 0.5 m/s. The smoothed command is the speed-up profile of Section 2: the speed loop gets a speed and an acceleration it can follow, and its largest command is 3.10 N, never at the limit.

Why 0.25 m/s^2. From rest the drive can give 0.5 m/s^2, but the drag eats into the 4 N as the speed grows. At 0.5 m/s the drag takes 1 N:

$$(4 - 2 \times 0.5)/8 = 0.375 \text{ m/s}^2$$

So the 0.5 m/s^2 of the motion-limits planner (acceleration limit) is only reachable near rest. A limit of 0.25 m/s^2 stays below what the drive gives at every speed up to 0.5 m/s and leaves 1 N for the feedback: the feedforward at the end of the speed-up is 3 N. Setting limits a little below the physical ones is the advice of Section 5.1 again.

7.2 Deadband: send nothing the motors cannot do

Near the goal the planner may ask for speeds such as 0.015 m/s. The motors cannot turn that slowly against their own friction, and Nav2's documentation warns that such small commands can damage hardware controllers "if that speed cannot be achieved due to stall torque". A command the robot cannot follow also leaves a speed error behind, which an integral would wind up on, the worry behind Autoware's rule of not integrating while stopped (Section 5.3). A band of small values around zero inside which the command is replaced by 0 is a **deadband** (G-3084). Nav2's smoother rounds every output whose size is under `deadband_velocity` to 0 before sending it; we use 0.02 m/s.

Figure 13 runs the planner's raw commands through the smoother, one frame per three cycles. Watch the blue output climb in steps of 0.0125 m/s instead of jumping, and in the middle panel watch the smoother's own value settle at 0.015 m/s at t = 5.4 s while the command it sends stays at 0. The bottom panel shows the force the speed loop asks for: 11 N with the raw jump, at most 3.10 N with the smoothed commands.

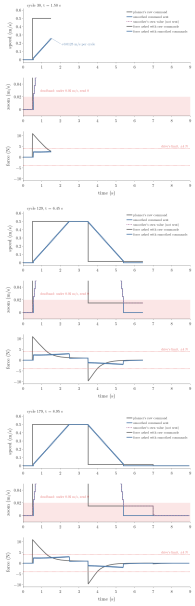


Figure 13: Nav2's velocity smoother, run at 20 Hz with an acceleration limit of 0.25 m/s^2, a top speed of 0.6 m/s and a deadband of 0.02 m/s; one frame per three cycles. Top: the planner's raw commands (grey: 0, then 0.5 m/s from 0.5 s, 0.015 m/s from 3.5 s, 0 from 7 s) and the smoothed command sent (blue). Middle: the same near zero; the smoother's own value (purple dotted) reaches 0.015 m/s at 5.4 s, inside the deadband (red band), so 0 is sent. Bottom: the force the speed loop asks for: up to 11 N with the raw commands (0.77 s at the 4 N limit after the jump, red dotted), at most 3.10 N with the smoothed ones

The node that applies these limits to every velocity command before it reaches the robot's base is a **velocity smoother** (G-3085). Nav2's (`nav2_velocity_smoother`; parameters in `nav2_bringup/params/nav2_params.yaml`) clamps each command to `max_velocity`, limits its change with `max_accel` and `max_decel` (2.5 m/s^2 each by default), and applies `deadband_velocity`; its README gives the purpose as reducing "wear-and-tear on robot motors and hardware controllers" from jerky commands. Autoware's longitudinal controller has the same kind of limits on acceleration and jerk (`max_acc`, `max_jerk` in its parameter file).

The speed commands now stay within what the drive can do. The last failure of the Overview is the heading: one strip of carpet knocks it 11.1° off course.

8. Cascaded loops: a fast inner loop under a slower outer loop

> **Key point:** In a cascade the outer loop's output is the inner loop's reference. A fast inner loop on a quick sensor, here the gyro, corrects a disturbance before the heading moves much and can be tuned on its own; on a big turn it holds the robot to the turn rate the outer loop asks for, so the robot brakes in time.

8.1 One loop for the heading, or two

The sixth failure of the Overview is the heading. The robot's heading obeys Newton's second law for rotation (G-2957): the torque (G-2808) changes the yaw rate, the turning speed, by the torque divided by the yaw moment of inertia (G-2955), 0.1 kg · m^2; the drive gives at most 0.2 N · m. A gyro measures the yaw rate, and the heading is the yaw rate added up over time.

RO-060's heading loop is one PID controller (G-2965) that sets the torque from the heading error, with gains of 0.2 N · m per rad, 0.05 N · m per rad · s on the error's running total and 0.283 N · m per rad/s on the gyro's yaw rate. When one wheel rolls onto a strip of carpet, the extra drag on that wheel turns the robot with a torque of 0.05 N · m. The PID can only answer once the heading error has grown, and with gains this small it answers slowly: the heading swings 11.1° off course and is back within 0.5° only 10.6 s after the carpet (Figure 16, Section 8.3).

A cascade splits the loop in two (Figure 14):

- the first loop turns the heading error into a wanted yaw rate, 1 rad/s per rad of error, capped at 1 rad/s;
- the second loop turns the yaw-rate error, measured by the gyro, into torque, with a PI.

Nesting loops so that the outer loop's output is the inner loop's reference is **cascade control** (G-3086) (FBS §1.5). The loop that sets the reference, here the heading loop, is the outer loop; the loop that receives it, here the yaw-rate loop, is the inner loop. Together they are the **inner and outer loops** (G-3087). Two loops need two sets of gains, so the cascade cannot reuse RO-060's three; Section 8.2 chooses them one loop at a time.

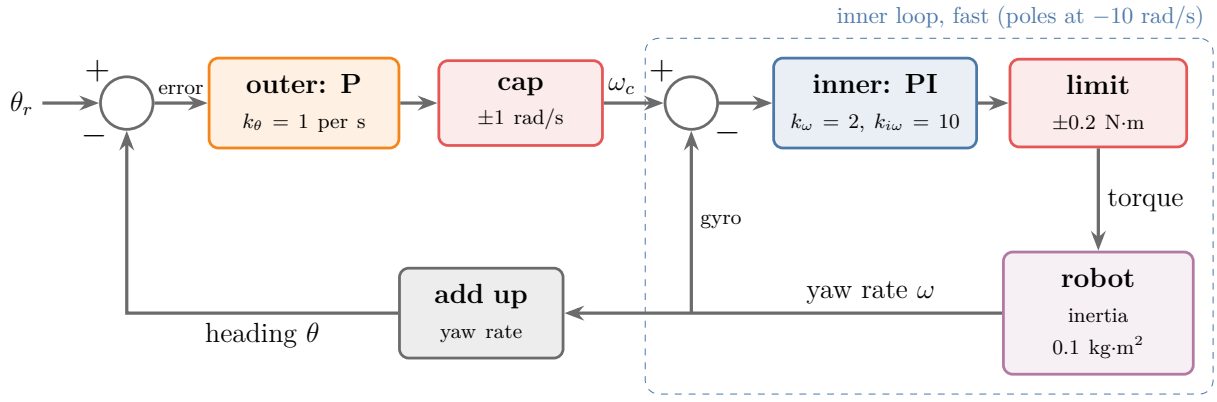


Figure 14: The heading cascade. The heading reference θ_r minus the heading θ gives the error; the outer loop, a P controller with k_θ = 1 per s, turns it into a yaw-rate command ω_c, capped at ±1 rad/s. The yaw-rate command minus the gyro's yaw rate ω enters the inner loop, a PI with k_ω = 2 and k_iω = 10, whose torque passes a ±0.2 N · m limit into the robot (yaw inertia 0.1 kg · m^2). The yaw rate is added up into the heading. The dashed box is the fast inner loop, with poles at −10 rad/s

8.2 Inner loop first, then the outer loop

A cascade is tuned from the inside out: first the inner loop, then the outer loop on top of it (FBS §1.5). The inner loop sees only the bare inertia, with no drag. With:

- J, the yaw inertia, 0.1 kg · m^2;
- ω_c and ω, the yaw-rate command and the yaw rate, in rad/s;
- ε, the yaw-rate error, the command minus the yaw rate, in rad/s;
- k_ω and $k_{i\omega}$, the inner PI's gains, 2 N · m per rad/s and 10 N · m per rad.

For a fixed command, the torque changes the yaw rate, the PI sets the torque from the error and its running total, and the error falls exactly as fast as the yaw rate rises. Our numbers first:

$$0.1\,\frac{d\omega}{dt} = 2\,\varepsilon + 10\int \varepsilon\,dt$$

$$0.1\,\frac{d^2\omega}{dt^2} = 2\,\frac{d\varepsilon}{dt} + 10\,\varepsilon$$

$$-0.1\,\frac{d^2\varepsilon}{dt^2} = 2\,\frac{d\varepsilon}{dt} + 10\,\varepsilon$$

$$0.1\,\frac{d^2\varepsilon}{dt^2} + 2\,\frac{d\varepsilon}{dt} + 10\,\varepsilon = 0$$

The same in symbols:

$$J\,\frac{d\omega}{dt} = k_\omega\,\varepsilon + k_{i\omega}\int \varepsilon\,dt$$

$$J\,\frac{d^2\omega}{dt^2} = k_\omega\,\frac{d\varepsilon}{dt} + k_{i\omega}\,\varepsilon$$

$$J\,\frac{d^2\varepsilon}{dt^2} + k_\omega\,\frac{d\varepsilon}{dt} + k_{i\omega}\,\varepsilon = 0$$

Its characteristic equation (G-2993), whose roots are the inner loop's poles (G-3000), with s the unknown root:

$$0.1\,s^2 + 2\,s + 10 = 0$$

$$0.1\,(s^2 + 20\,s + 100) = 0$$

$$0.1\,(s+10)^2 = 0$$

Both inner poles sit at -10 rad/s, a double pole, so the inner loop is critically damped (G-2995): its error dies away without oscillating, within a few tenths of a second. FBS Example 12.9 makes its inner loop 10 times faster than the outer one, and so do we: the outer gain of 1 per s gives the outer loop a time constant of 1 s.

Why the outer loop can ignore the inner one. Seen from the outer loop, a fast inner loop makes the yaw rate equal its command at once. The heading then changes at the commanded rate, 1 rad/s at the cap, and in proportion to the error below it: a first-order system (G-2987) with a time constant (G-2988), the time to cover 63 percent of the way, of 1 s. For the 90° turn, the error is above 1 rad, so the rate is capped, until the heading reaches 90° minus 57.3°:

$$1.5708 - 1 = 0.5708 \text{ rad}$$

$$0.5708/1 = 0.571 \text{ s at the cap}$$

Below the cap the heading's rate is 1 per s times the error, so the error shrinks by the same fraction every moment. With:

- t, the time since the rate left the cap, in s;
- θ_e, the heading error, in rad, 1 rad when the rate leaves the cap.

$$\theta_e = 1 \times \exp(-t)$$

The error reaches 2°, 0.0349 rad, when:

$$\exp(-t) = 0.0349$$

$$t = \ln(1/0.0349) = 3.355 \text{ s}$$

With the time at the cap:

$$0.571 + 3.355 = 3.93 \text{ s}$$

Figure 15 plays the real turn, inner loop included. Watch the blue cascade first. In the middle panel the inner loop catches its 1 rad/s command within 0.48 s, with the torque at its 0.2 N · m limit for the first 0.46 s. In the bottom panel the heading comes within 2° of 90° after 4.22 s, about 0.3 s after the 3.93 s of a perfect inner loop (grey dotted), and does not overshoot; the turn rate peaks at 1.015 rad/s, just above the cap. The inner loop settles

first; the outer loop barely notices it. Then watch the purple curve, RO-060's single PID: it overshoots the 90° by 20.8°, because its integral keeps adding up the error during the whole turn. (The red curve belongs to Section 8.3.)

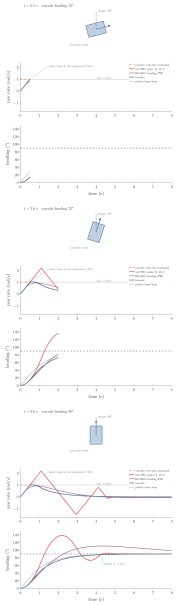


Figure 15: A 90° turn three ways, one frame per 0.1 s. Top: the cascade robot from above, turning towards the 90° target (grey dashed). Middle: yaw rate; the cascade's yaw-rate command from the outer loop (orange dashed), capped at 1 rad/s (grey dotted), and the yaw rate it makes from the gyro (blue), on its command after 0.48 s. Bottom: heading against the 90° target and against the heading a perfect inner loop would give (grey dotted). Blue, the cascade: within 2° after 4.22 s, no overshoot. Purple, RO-060's heading PID (0.2, 0.05, 0.283): overshoots by 20.8°. Red, one heading PID with the cascade's gains multiplied out (12, 10, 2), with clamping: spins at up to 2.19 rad/s and overshoots by 48.4°

8.3 The inner loop catches disturbances first

The carpet of Section 8.1, now with the cascade. The gyro sees the yaw rate change at once, long before the heading has moved much, and the inner loop's integral learns the 0.05 N · m, as the integral action of Section 3 learned the ramp's pull. Figure 16 shows the result: the cascade's yaw rate is back within 0.01 rad/s at t = 1.21 s, 0.21 s after the carpet, and its heading dips by only 0.21° before returning to 0. RO-060's PID swings 11.1° off.

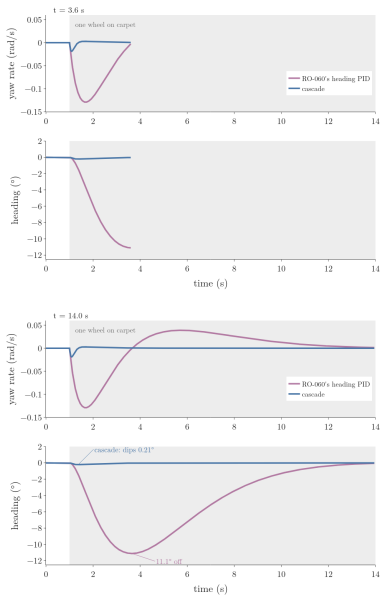


Figure 16: Driving straight, one wheel rolls onto carpet at t = 1 s (grey band) and turns the robot with a torque of -0.05 N · m; one frame per 0.15 s. Top: yaw rate from the gyro. Bottom: heading. RO-060's heading PID (purple) swings 11.1° off at t = 3.65 s and is back within 0.5° at 11.56 s. The cascade (blue) has its yaw rate back within 0.01 rad/s at 1.21 s and dips 0.21° before returning to 0

Why not just raise the single loop's gains? Multiplied out, the cascade's two loops give one torque rule on the heading error. Take the carpet case, where the target heading is 0, so the heading is minus the error and the yaw rate's running total is the heading. With:

- e, the heading error, in rad, and ω, the yaw rate, in rad/s;
- k_θ, the outer gain, 1 per s, and k_ω, $k_{i\omega}$, the inner gains, 2 and 10;
- τ, the torque, in N · m.

Our numbers first:

$$\tau = 2\,(1 \times e - \omega) + 10 \int (1 \times e - \omega)\,dt$$

$$\int \omega\,dt = -e$$

$$\tau = 2\,e - 2\,\omega + 10 \int e\,dt + 10\,e$$

$$\tau = 12\,e + 10 \int e\,dt - 2\,\omega$$

The same in symbols:

$$\tau = k_\omega(k_\theta e - \omega) + k_{i\omega} \int (k_\theta e - \omega)\, dt$$

$$\tau = (k_\omega k_\theta + k_{i\omega})\, e$$

$$+ k_{i\omega} k_\theta \int e\, dt - k_\omega\, \omega$$

The result is a single heading PID with gains 12, 10 and 2, and on the carpet it does what the cascade does: a dip of 0.209° against 0.212° (notebook).

The multiplying out used the carpet case, where the robot starts on its target heading. A turn starts 1.5708 rad away, and there the two differ: the red curve of Figure 15. The single loop asks for 18.8 N · m at once and keeps the full 0.2 N · m until its derivative part outweighs its proportional part:

$$12\, e - 2\, \omega = 0$$

$$\omega = 6\, e$$

So it starts braking only when the yaw rate is about six times the error: 0.35 rad from the target, already turning at 2.19 rad/s (notebook). It cannot stop in time and overshoots by 48.4°, even with clamping. The cascade's inner loop starts braking as soon as the yaw rate passes the rate the outer loop asks for: 1.18 rad from the target, at 1.02 rad/s (notebook). The 1 rad/s cap is not what saves the turn: without it the cascade peaks at 1.19 rad/s and still does not overshoot (notebook); the cap only keeps the turn rate under a set limit. The same gains are safe in the cascade because the outer loop asks for a turn rate that falls with the error and the inner loop holds the robot to it; the two-step tuning is what finds them.

8.4 Cascades in real stacks

The cascade's advantages, a simple plant for the outer loop, an early catch of disturbances, loops tuned one at a time and a place for limits, make nested loops common (FBS §1.5):

- an electric drive nests three loops: current inside velocity inside position (FBS §1.5, Figure 1.13); the robot's motor amplifier runs its own current loop to make the asked-for force (MR §11.1);
- **PX4** is "a standard cascaded control architecture" (its controller diagrams page, `controller_diagrams.md`): position (P), then velocity (PID), then attitude (P) and body rate (PID), with the velocity and rate commands capped between loops (`controller_diagrams.md`, `PositionControl.cpp`); its tuning guide tunes the rate loop first;

- **Apollo**'s longitudinal controller (`lon_controller.cc`) runs a station PID, on the distance along the path, whose output is a speed offset for a speed PID; the parameter file (`lon_based_pid_controller/conf/controller_conf.pb.txt`) sets both and a step of 0.01 s;
- ros2_controllers' `pid_controller` can be chained with "preceding or following controllers", so one controller's output becomes the next one's reference.

Figure 17 draws the PX4 and Apollo cascades.

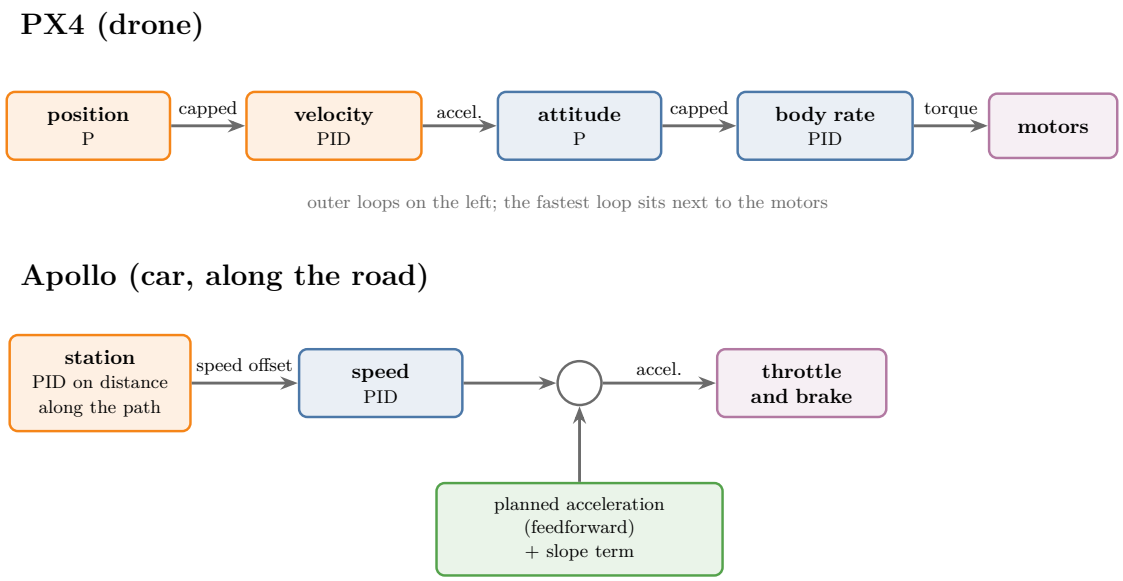


Figure 17: Cascades in two open-source stacks. PX4 (drone): position (P) → velocity (PID) → attitude (P) → body rate (PID) → motors, with the velocity and rate commands capped between loops; the fastest loop sits next to the motors. Apollo (car, along the road): station PID on the distance along the path → speed offset → speed PID → acceleration, plus the planned acceleration (feedforward) and a slope term → throttle and brake

The robot's own stack is a cascade too: Nav2's controller computes velocity commands 20 times a second, the velocity smoother of Section 7 limits them, and the robot's base runs the speed and yaw-rate loops of this Note at 100 Hz.

9. Summary

Problem (number)	Fix	Why it works	Result
trails the speed-up, ends 10 cm behind	feedforward	computes the force from the plan and the model before any error	0 cm on the model's floor, 5 cm on carpet
state feedback on a 1° ramp: 0.438 m/s	integral state	a settled integral state means zero error, whatever the model	0.5 m/s
wheel's push: swings 0.027 to 0.073 m/s	resonant term	an oscillator at 1 rad/s cancels a push at 1 rad/s	swing under 0.00001 m/s
held 3 s: 25.9 N asked, peak 1.04 m/s	clamping or back-calculation	the integral stops growing while the drive is saturated	peak 0.515 m/s, or no overshoot
dock pressed with 4 N	force control	at rest the drive force is the contact force	2 N
jump to 0.5 m/s asks 11 N	rate limiter, deadband	commands change no faster than the drive can follow	at most 3.10 N
carpet swings the heading 11.1°	cascade	the gyro loop answers the torque at once and holds the turn rate the outer loop asks for	dip of 0.21°, 90° turn without overshoot

- The PI loop failed in six ways on the robot, and each now has a fix: the lag of 10 cm becomes 0 with feedforward, the wheel's swing of 0.023 m/s becomes under 0.00001 m/s with a resonant term, the 1.04 m/s after holding becomes 0.515 m/s with clamping, the 4 N on the dock becomes 2 N with force control, the 11 N jump becomes at most 3.10 N with a rate limiter, and the carpet's 11.1° becomes 0.21° with a cascade, because each fix removes the cause the opening named.
- **Feedforward** (G-3070) sends the force the plan needs, mass times acceleration plus drag, before any error, so the robot no longer waits for an error to move; feedback still has to remove what the model gets wrong, as on carpet.
- **Integral action in state feedback** (G-3073) adds the running total of the error as a state, so a settled loop has zero error for any model and any constant disturbance, which is why the PI held 0.5 m/s on the ramp.
- The **internal model principle** (G-3074) says the controller must contain a model of what it cancels, so a **resonant term** (G-3075) at the wheel's 1 rad/s cancels the wheel's push where the integral could not.
- **Actuator saturation** (G-3076) breaks the loop, and **integrator windup** (G-3077)

follows because the integral keeps growing on an error the drive cannot remove; the overshoot comes from unwinding it.

- **Clamping** (G-3079) stops the integration while the command is saturated and the error pushes further, and **back-calculation** (G-3080) feeds the shortfall into the integrator; both are **anti-windup** (G-3078), and PX4, Apollo and ros2_control offer both.
- **Force control** (G-3082) replaces **motion control** (G-3081) when the task is to press, because a speed error against a fixed dock never goes away.
- A **velocity smoother** (G-3085) with a **rate limiter** (G-3083) and a **deadband** (G-3084) keeps commands within what the drive can follow, so saturation and windup are not invited in the first place.
- **Cascade control** (G-3086) puts a fast **inner loop** (G-3087) on the gyro under the heading loop, so disturbances are caught early and each loop is tuned on its own; the same gains in one loop overshoot a 90° turn by 48.4°, because one loop starts braking only when the turn rate is about six times the error, while the cascade brakes as soon as the rate passes the outer loop's command.

The robot can now follow its plan, recover from a hand that holds it, press its dock gently and turn within its limits. Still missing: every gain here, from RO-060's 20 and 10 to the back-calculation's 1 per s, was simply given; choosing gains on a real robot, with noisy sensors and unknown lags, is PID tuning.

10. Sources

Built from

- MATLAB, "Anti-windup for PID control | Understanding PID Control, Part 2", YouTube, https://www.youtube.com/watch?v=NVLXCwc8HzM (MATLAB, Anti-windup for PID control)
- MATLAB, "What Is Feedforward Control? | Control Systems in Practice", YouTube, https://www.youtube.com/watch?v=FW_ay7K4jPE (MATLAB, What Is Feedforward Control?)
- Åström, K. J. and Murray, R. M. (2021). *Feedback Systems*, 2nd ed. §1.2 What is feedforward, §1.5 Feedback properties (cascaded loops, Figure 1.13), §7.2 Stabilization by state feedback, §7.4 Integral action, §8.5 State space controller design, §11.1 Basic control functions, §11.4 Integral windup (Examples 11.5, 11.6, Figure 11.11), §12.4 Feedforward design, §12.6 Design example (Example 12.9). Free: https://fbswiki.org (FBS)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. §11.1 Control system overview, §11.3.1 Motion control of a single joint, §11.5 Force control. Free preprint: http://modernrobotics.org (MR)
- Schenato, L. *Control Laboratory*, lecture notes 14 and 15, "Internal model principle", University of Padova, 2016, automatica.dei.unipd.it, CL_2016_Lecture_15.pdf (Schenato, lectures 14 and 15)
- Beauregard, B. "Improving the Beginner's PID: Reset Windup", 2011, brettbeauregard.com blog
- Nav2, in https://github.com/ros-navigation/navigation2: nav2_velocity_smoother /

src / velocity_smoother.cpp and its README.md, the parameters in nav2_bringup / params / nav2_params.yaml, and the configuration guide https://docs.nav2.org (Velocity Smoother); read for the algorithm, the defaults and the purpose of the deadband

- Autoware, in https://github.com/autowarefoundation/autoware_universe: control / autoware_pid_longitudinal_controller / src / pid.cpp, src / pid_longitudinal_controller.cpp, config / autoware_pid_longitudinal_controller.param.yaml and README.md; read for the feedforward, the integral limit and the no-integration-while-stopped rule
- Apollo, in https://github.com/ApolloAuto/apollo: modules / control / controllers / lon_based_pid_controller / lon_controller.cc and conf / controller_conf.pb.txt, and modules / control / control_component / controller_task_base / common / pid_controller.cc, pid_IC_controller.cc and pid_BC_controller.cc; read for the station and speed cascade, the feedforward and the two anti-windup methods
- PX4, in https://github.com/PX4/PX4-Autopilot: src / modules / mc_pos_control / PositionControl / PositionControl.cpp, src / lib / rate_control / rate_control.cpp, docs / en / flight_stack / controller_diagrams.md and docs / en / config_mc / pid_tuning_guide_multicopter.md; read for the cascade, the feedforward and the anti-windup
- ros2_control: control_toolbox / include / control_toolbox / pid.hpp and control_toolbox / src / pid.cpp in https://github.com/ros-controls/control_toolbox, and pid_controller / src / pid_controller.yaml and pid_controller / doc / userdoc.rst in https://github.com/ros-controls/ros2_controllers; read for the anti-windup strategies, the feedforward gain and chaining

Other references

- MATLAB, "Important PID Concepts | Understanding PID Control, Part 7", YouTube, https://www.youtube.com/watch?v=tbgV6caAVcs
- NPTEL (IIT Kharagpur), *Industrial Automation and Control*, course 108105063, lectures 14 "Implementation of PID Controllers", 15 "Feedforward and Ratio Control" and 17 "Cascade Control", https://archive.nptel.ac.in/courses/108/105/108105063

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Feedforward (G-3070)	A control force computed from the plan and the robot's model, such as mass times the planned acceleration plus drag ($ma_r + bv_r$), and sent before any error appears; it lets the robot follow a changing plan without first falling behind, while feedback removes what the model gets wrong.

Term	Meaning
Disturbance feedforward (G-3071)	Adding to the command the force that cancels a disturbance that can be measured, such as the 1.37 N a tilt sensor says a 1° ramp pulls back with; it acts before the disturbance changes the speed, but only for disturbances a sensor can see.
State feedback (G-3072)	A controller that multiplies the system's state by a gain and adds a gain times the reference, $u = -kv + k_f r$ for a speed loop; it lands on the reference only if the gains match the true model and no constant disturbance acts.
Integral action in state feedback (G-3073)	Adding the running total of the error, z with $dz/dt = r - v$, as an extra state of the controller and feeding it back; a settled loop then has zero error for any model and any constant disturbance, because z stops changing only when the error is zero.
Internal model principle (G-3074)	The rule that a controller can cancel a disturbance, or follow a reference, with zero error only if it contains a model of that signal: an integrator for a constant, an oscillator at the same frequency for a sine.
Resonant term (G-3075)	A controller part that oscillates by itself at a chosen frequency and is driven by the error; added to a PI, it cancels a disturbance at that frequency, such as an out-of-round wheel's push once per turn, because it can keep producing the cancelling wave while the error is zero.
Actuator saturation (G-3076)	An actuator stuck at its limit while the command asks for more, such as a drive giving its 4 N when 25.9 N is asked; while it lasts the force no longer depends on the error, so the feedback loop is broken.
Integrator windup (G-3077)	The integral term growing while the actuator is saturated, because the error it cannot remove persists, and then keeping the actuator saturated after the error changes sign; it causes large overshoot, such as 1.04 m/s after a robot held for 3 s is let go.

Term	Meaning
Anti-windup (G-3078)	Any change to a controller with integral action that keeps the integral from growing while the actuator is saturated, so the command leaves the limit as soon as the error changes sign; clamping and back-calculation are the two common kinds.
Clamping (conditional integration) (G-3079)	Anti-windup that stops the integration while the controller's output is saturated and the error has the same sign as the command; it keeps the integral from piling up, and some sources use the same word for limiting the integral term's value.
Back-calculation (G-3080)	Anti-windup that feeds the gap between the force the actuator gives and the command, $u - c$, into the integrator through a gain k_{aw}; it pulls the command back towards the limit while saturated and does nothing otherwise.
Motion control (G-3081)	Control of where a robot goes and how fast, keeping the motion the same whatever forces push on it; against an object that does not move its error never goes away, so it pushes as hard as the actuator allows.
Force control (G-3082)	Control of the force a robot applies to its surroundings, set directly and corrected by a force sensor if there is one; used when the task is to press rather than to move, such as holding 2 N against a charging dock.
Rate limiter (G-3083)	A block that lets a signal change by at most a set amount per second, such as a speed command by 0.25 m/s every second (0.0125 m/s per 20 Hz cycle); it keeps commands within what the actuator can follow, so they do not saturate it.
Deadband (G-3084)	A band of small values around zero inside which a command is replaced by 0, such as speed commands under 0.02 m/s; it keeps the robot from being asked for motions its motors cannot start.

Term	Meaning
Velocity smoother (G-3085)	A node between the planner and the robot's base that clamps each velocity command to the speed limits, limits its rate of change and applies a deadband, such as Nav2's `nav2_velocity_smoother`; it turns jumpy planner output into commands the drive can follow.
Cascade control (G-3086)	Nesting feedback loops so that the outer loop's output is the inner loop's reference, such as a heading loop that commands the yaw rate for a gyro loop; it lets each loop be designed in turn, catches disturbances early and offers a place for limits.
Inner and outer loops (G-3087)	The two levels of a cascade: the outer loop sets the reference, the inner loop, usually faster and on a quicker sensor, follows it; the inner loop is tuned first and made about 10 times faster, so the outer loop can treat it as perfect.
PID controller (G-2965)	A controller whose command is the sum of a proportional, an integral and a derivative part of the error, K_p e + K_i (integral of e) + K_d de/dt; the commonest feedback controller, it reacts to the error now, its past and its trend; with K_d = 0 it is a PI controller, with K_i = 0 a PD controller.
Step response (G-2981)	The output of a system after a step input, such as the robot's heading after its command jumps to 0.5 rad; its rise time, overshoot, settling time and steady-state error describe how well a loop follows a command.
Pole (G-3000)	A value of s where a transfer function's denominator is zero, so the fraction blows up there, such as -2 (twice) for the PD heading loop; each pole adds a term e^(pt) to every reply, so its real part says fade or grow and its imaginary part how fast it swings.
Integral action (G-2963)	A part of the command in proportion to the error's running sum (its integral over time); the sum grows while any error remains, so it removes the error a steady load leaves, at the cost of overshoot and, if too large, instability.

Term	Meaning
Integral gain (G-2964)	The factor K_i by which integral action multiplies the error's running sum, such as 0.05 N m per rad s; larger values remove a steady error sooner but swing more, and above a limit make the loop unstable.
Newton's second law (G-2809)	The net force on a body equals its mass times its acceleration; it gives the force needed to change an object's speed, which the quasistatic assumption neglects.
State x_t (G-2314)	The quantities a robot keeps track of at time t because they decide what happens next, such as its pose, its speeds and moving things around it; it is what localization and filtering estimate.
Rolling without slipping (G-2290)	A wheel moving so that its contact point does not skid: it moves forward by its radius times the angle it turns and never slides sideways, which is what lets wheel turns predict motion.
Plant (G-2945)	The system being controlled, such as the robot's body rolling with its mass and drag; its output is the quantity we want to control, and actuator, plant and sensor are often lumped together under this name.
Actuator (G-2948)	The part of a feedback loop that turns the controller's command into a physical action on the plant, such as motors producing a force of at most 4 N; its limits bound what the controller can do.
Newton's second law for rotation (G-2957)	The net torque on a body equals its moment of inertia times its angular acceleration, tau = I alpha; it gives how fast a torque spins a body up, the turning form of F = ma.
Torque (G-2808)	The turning effect of a force about a point: the force times its distance from the point, measured square to the force's line; comparing torques decides whether a pushed object tips.

Term	Meaning
Moment of inertia (G-2955)	How hard a body is to spin up or slow down about an axis: the sum of each bit of mass times its distance from the axis squared, such as 0.1 kg m^2 for the robot about its vertical axis; it plays for turning the part mass plays for driving.
Characteristic equation (G-2993)	The polynomial equation found by trying a solution e^(st) in a linear differential equation, such as $s^2 + 0.8\ s + 4 = 0$ for the heading error; its roots say whether the solution fades, swings or grows.
Critically damped (G-2995)	A second-order system with damping ratio exactly 1: its characteristic equation has one double real root, so it gives the fastest step response that does not overshoot, such as the turn with K_p = K_d = 0.4 settling in 2.92 s.
First-order system (G-2987)	A system whose output follows a differential equation with only a first derivative, such as the robot's speed under P control; it closes the same fraction of its remaining gap in every time constant and can never overshoot.
Time constant (G-2988)	The time a first-order system takes to close 63 % of the gap to its final value, m/(K_v + b) = 1 s for the robot's speed loop; it fixes the rise time (about 2.2 time constants) and the 2 % settling time (about 4).